CLOUD & DEVOPS

Blue-Green Deployments Visual Guide

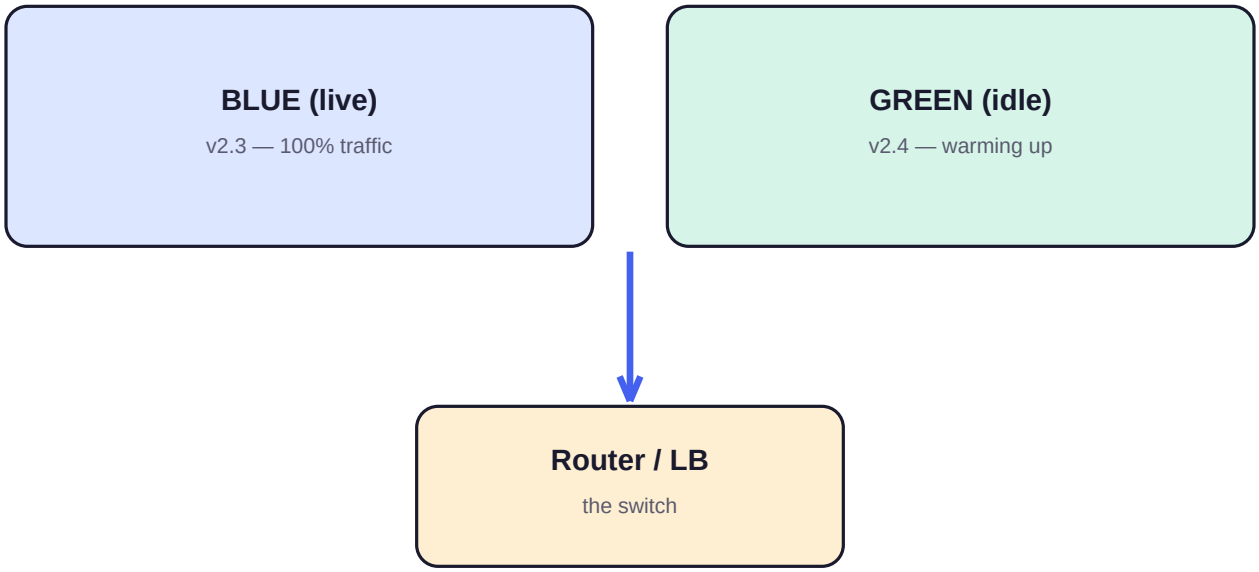
Two identical environments —
cut traffic over, roll back in seconds.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/5

Two Identical Environments

Blue serves traffic; green waits for the next release.



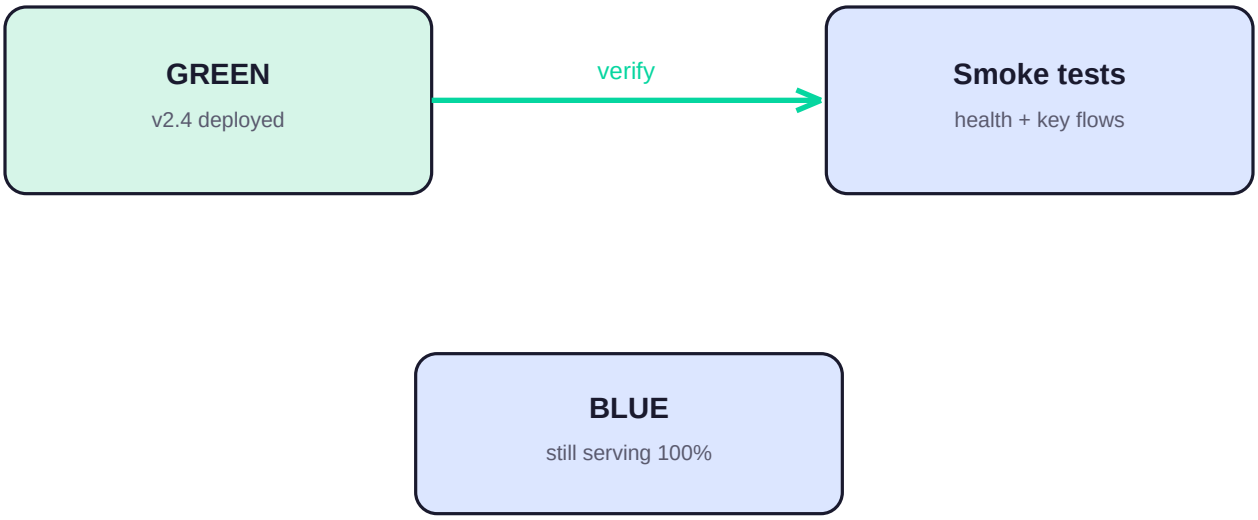
Identical means identical: same size, same config, same data shape.

Cost: you run 2x capacity during the switch — the price of instant rollback.

STEP 2/5

Deploy to Green, Verify in Peace

The new version goes live nowhere until you say so.



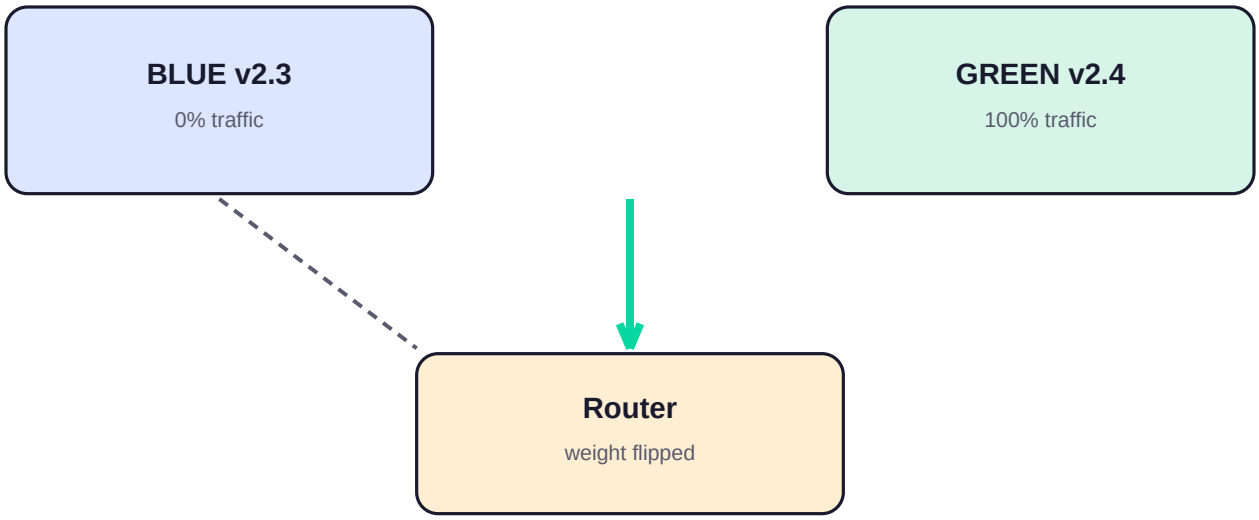
Green gets real warm-up: caches fill, JIT compiles, connections pool — no cold traffic shock.

- Run the full smoke suite against green's real endpoint.
- Shadow traffic (copy of live requests) finds issues smoke tests miss.
- Green stays dark to users until the cutover — zero user impact while you verify.

STEP 3/5

The Cutover: Flip the Router

One config change moves 100% of traffic in seconds.



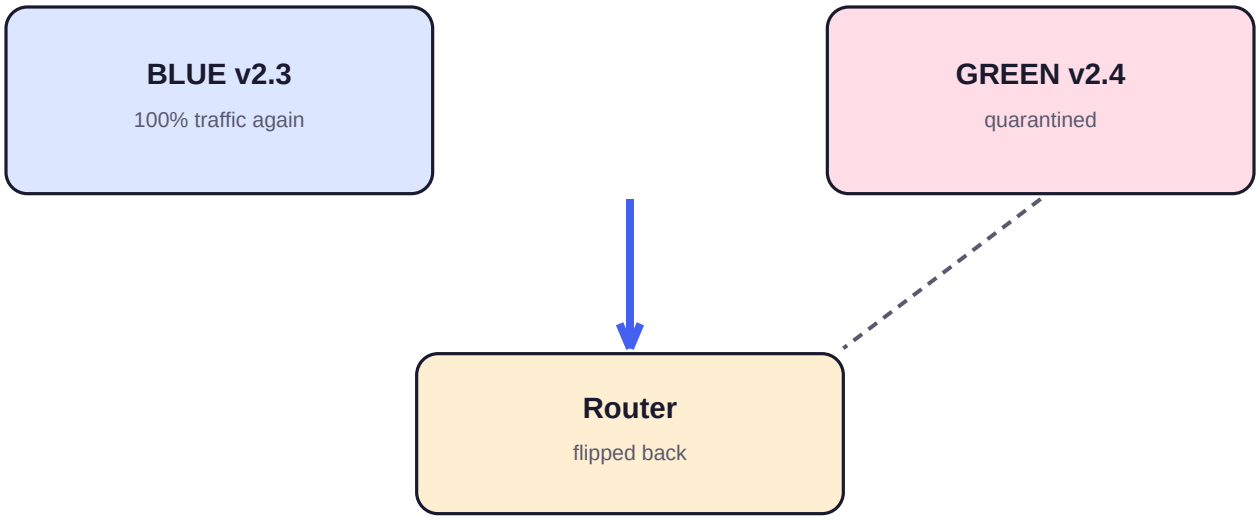
Cutover is a router/LB/DNS weight change — atomic, fast, and observable.

Watch error rates and latency for 10–30 min after the flip. Keep blue warm — don't tear it down yet.

STEP 4/5

Instant Rollback

Something's wrong? Flip the switch back.



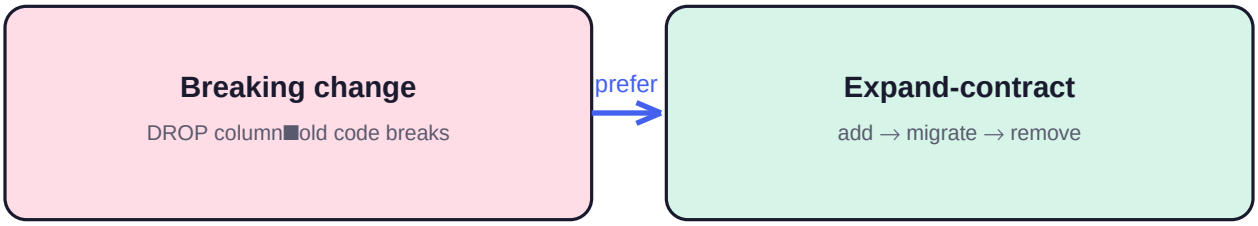
Rollback needs no redeploy — the old version is still warm and waiting.

Decide the rollback trigger BEFORE the cutover: error rate > X% for Y minutes → flip back, no meeting needed.

STEP 5/5

The Hard Part: The Database

Code can flip instantly; schemas can't.



- Expand: add the new column/table; both versions work against it.
- Migrate: backfill data; deploy the version that uses the new shape.
- Contract: only AFTER old code is gone, drop the old column. Three deploys, zero downtime.

Shared database = shared fate. Blue-green without expand-contract is just downtime with extra steps.

STEP ✓I✓

Key Takeaways

Blue-Green Deployments — Remember

- Two identical environments: one live, one staging the next release.
- Verify green fully (smoke + shadow traffic) before any user sees it.
- Cutover is a router flip — seconds, atomic, observable.
- Rollback is the same flip in reverse: no redeploy, old version still warm.
- Databases need expand-contract migrations — code flips fast, schemas don't.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev